

Relatório trabalho 2

Guilherme Portas

Nesse trabalho, devemos implementar um Benchmark para acesso à memória, utilizando a linguagem de programação C, sendo conhecida por sua eficiência e portabilidade, permitindo os programadores a acessar e manipular diretamente a memória do computador, além de ser uma das linguagens mais próximas do Assembly, porém ainda com um bom equilíbrio entre recursos de alto e baixo nível. O objetivo é investigar, na prática, como diferentes padrões de acesso à memória afetam o tempo de execução de um programa, relacionando a localidade, cache e hierarquia de memória discutidos em aula.

Sendo assim, a implementação que devemos criar deve alocar dinamicamente um vetor com ao menos 10^6 elementos, nós teremos que medir o tempo total para acessar todos os elementos do vetor de forma sequencial e randômica e repetir esse experimento com três tamanhos de vetores diferentes.

Para medir o tempo de acesso de todos os elementos, usarei a biblioteca "time.h", que contém funções para obter e manipular informações sobre a data e o tempo. Uma das funções para medir a diferença entre dois tempos é a função "clock_gettime", que permite obter o tempo de diferentes fontes. Porém ela não está disponível para o Windows, que é o sistema operacional que eu uso. Então, decidi usar a função "clock", porém, essa obtém o tempo da CPU, que é medido em segundos por programa. Para converter esse tempo para segundos, temos que pegar a diferença entre um clock e outro e dividir pelo identificador "CLOCKS_PER_SEC", indica quantos "clocks" (a frequência na qual o processador opera) são executados a cada segundo.

No arquivo ".c" criado para esse trabalho, eu criei uma função chamada de "Benchmark", que cuida da execução dos testes, obtém como parâmetros a quantidade de elementos do vetor e o tipo de operação (sequencial ou randômico) e retorna o tempo total medido na execução. Nela, criamos uma array principal com a quantidade de elementos necessárias e atribuímos a cada posição um valor diferente. Caso a operação seja para teste de acessos randômicos, criamos uma segunda array, igual à primeira, e aleatorizamos as posições dos seus valores com uma função especial chamada de "Shuffle", então, usando esses valores sequencialmente, acessamos a array principal aleatoriamente.

Para forçar o compilador a utilizar a memória para obter os valores, somarei todos os valores da array em uma variável, e se eu não utilizá-la no código, o compilador a ignora a fim de evitar redundâncias e melhorar a eficiência, porém nós queremos verificar o tempo dessas execuções, então ao fim da função, imprimo o valor da soma no terminal, o que evita retornar dois valores em uma lista, que é mais complicado em C. Além disso, também libero a memória dos vetores para evitar "Memory Leak", que é quando o programa não libera corretamente a memória quando ela não é mais necessária.

Com isso feito, na função "Main", executo todos os testes necessários com os valores de 10^5 , 10^6 e 10^7 e imprimo a quantidade de tempo obtida no terminal. É possível perceber que até 10^5 valores, não existe quase nenhuma diferença, com execuções quase instantâneas. Com 10^6 valores, obtenho 0 segundos no acesso sequencial, porém com 0,004 segundos no acesso randômico. Já com 10^7 , o tempo sequencial é de 0,003 e o randômico de 0,031. Em média, o acesso sequencial demorou 0,001 segundos e o aleatório precisou de 0,012 segundos, aproximadamente.

	Quantidade de elementos →			
Tipo de acesso v				
	10 ⁵	10 ⁶	10 ⁷	Média
Sequencial	0.000000	0.000000	0.003000	0.001000
Randômico	0.000000	0.004000	0.031000	0.011667
Soma	4999950000	499999500000	49999995000000	

```
==== Benchmark de acesso ah memoria em C ====
```

```

    Testando com 100000 numeros
Soma: 4999950000
Tempo do acesso sequencial: 0.000000
Soma: 4999950000
Tempo do acesso randomico: 0.000000

```

```

    Testando com 1000000 numeros
Soma: 499999500000
Tempo do acesso sequencial: 0.000000
Soma: 499999500000
Tempo do acesso randomico: 0.004000

```

```

    Testando com 10000000 numeros
Soma: 49999995000000
Tempo do acesso sequencial: 0.003000
Soma: 49999995000000
Tempo do acesso randomico: 0.031000

```

```

Media do acesso sequencial: 0.001000
Media do acesso randomico: 0.011667

```

Como se pode observar nos testes realizados, o acesso randômico tende a ser mais demorado que o acesso sequencial, as vezes até 10 vezes mais. Mas por quê? Para entender isso, devemos primeiro analisar como o processador obtém os dados da memória.

Para isso, vamos dar uma olhada na estrutura da CPU. Ela é composta de diversos componentes, porém os principais são: a ALU ("Arithmetic Logic Unit", responsável por executar cálculos matemáticos e operações lógicas, no qual obtém suas entradas de registradores e entrega a saída das operações), a CU ("Control Unit", direciona a operação da CPU ao buscar, decodificar e executar instruções), os registradores (são locais de armazenamento pequenos e de alta velocidade dentro da CPU que temporariamente armazenam dados de instruções que a CPU está processando) e a memória cache.

Essa última serve como uma ponte de velocidade entre a CPU e a memória principal e é dividida em diferentes níveis, L1, a menor, porém com a velocidade mais rápida de até 1 nanosegundo, a L2, rápida com tamanho médio e as vezes também existe a L3, mais lenta, porém maior, e por fim, a memória RAM, a mais lenta de todas, porém também é a com mais capacidade de armazenamento. Essa estrutura permite que o acesso aos dados mais frequentes sejam executados extremamente rápidos, além de otimizar o acesso para outros dados menos utilizados ao compensar as velocidades mais baixas das memórias maiores, trazendo os dados para as caches mais rápidas.

Sendo assim, uma boa parte da performance resultante desse teste se deve a essa estrutura de cache. Os dados são primeiros acessados da memória RAM principal e levados para a cache, onde serão acessados de forma mais rápida posteriormente, então quando o compilador pedir o próximo valor, ele já está na cache mais rápida, então obtém o valor desse local. Mas o que eu tinha explicado, ele só trazia um único valor para o

cache. A memória foi projetada para transferir dados em blocos, então além de trazer o dado necessário, a CPU também traz para a cache vários bytes adjacentes, permitindo que a maioria dos dados sejam acessados quase instantaneamente. Isso é chamado de localidade espacial. Sendo assim, diversos dados já estarão em cache, sem a necessidade de buscar cada valor na memória principal, que é mais lenta.

Além desses mecanismos de otimização, a CPU também tem um outro chamado de “hardware prefetching”, no qual analisa os padrões de acesso da memória e faz uma previsão do próximo elemento que vai ser acessado e já o envia para a cache, adiantando o acesso antes mesmo da instrução ser executada.

Devido a esses dois últimos mecanismos de otimização, o acesso sequencial dos dados tende a ser mais rápido do que o randômico. No acesso sequencial, os dados possuem um padrão bem definido, então o “hardware prefetching” entra em ação para otimizar os próximos acessos, além disso, como os dados da memória são importados para a cache em blocos por causa da localidade espacial, após o primeiro acesso, os próximos dados são acessados diretamente da cache até a finalização do bloco, sendo necessário a importação dos próximos blocos, no qual a CPU já teria previsto. Já no acesso randômico, não existe nenhum padrão de acesso em que a CPU consegue prever, portanto nem a localidade espacial e nem o “hardware prefetching” conseguem agir para otimizar o acesso. A localidade espacial até funciona, porém o próximo acesso pode estar em um bloco totalmente diferente, que diminui a eficiência de acesso.

Ademais, com dados sequenciais, a CPU traz diversas páginas de dados sequenciais para a cache nas quais ela julga que será utilizada pelo programa. Essa técnica se chama “sequential prefetching”, mais um motivo para o acesso sequencial ser mais rápido que o randômico. Com isso, conclui-se o relatório desse trabalho.